The Latent Monitor Audit: What We Built, How It Works, and What It Found

A technical report on the inference-only study of continuous-thought reasoners

Muhammad Jon Raza
Project repository: `0x0shephard/latent-reasoning`
Research ledger §108–§136, October 2026

Abstract

Some language models solve math word problems without writing their steps. Instead they produce six hidden vectors, called "thoughts", and then the answer. Several groups want to watch these models by training a small classifier (a "probe") to read the thoughts. This report explains what we did to test whether that works. We built an audit that needs no training and asks three separate questions about each thought: does it *carry* a value into the answer, does the model *need* it, and when a probe reads a value from it, does the model's behaviour *confirm* that reading. We ran the audit on eight released models, five datasets and 39 preregistered Kaggle runs that reproduce byte for byte. The short version of what we found: a probe can read a thought well even when the model never uses it, and the model can use a thought it does not strictly need. Below 3B, a thought carries a value only on problems with three or more steps. One or two thoughts do the carrying and pass the value to the next position's memory. The last thought is readable and never used. Probes also read facts about the question from every thought, so a probe can look accurate while reading nothing the model computed. The thoughts predict wrong answers no better than the model's own confidence. The one practical gain is not in the thoughts: send the quarter of problems the model is least sure about to its own written solution, and accuracy rises by 3.7, 7.5 and 10.2 points at 1B, 3B and 8B. We also report a new experiment on three models: a one-vector write into the carrying thought steers 13–21% of correct answers to a value an attacker chose (control 0.2–1.5%); at 1B the same write into the unused thought sets off a probe on 100% of cases while changing nothing; and, against our prediction at every size, the thought after the written one keeps the old value, so a detector there catches the attack at 0.92, 0.97 and 0.98 AUROC at 1B, 3B and 8B. Finally we trained a published latent-reasoning safety monitor's model with its authors' code and tested it causally: its probe reads the armed state perfectly at every latent position (not only the early ones), an inert-trigger control shows that reading is not the trigger token's footprint, and swapping, removing or editing the probed latents changes the model's behaviour by nothing at all. The monitor detects the trigger; it does not read anything the model acts on. Following the behaviour to where it lives, we find that the trigger token's own memory entry is not read by the answer either, that a perfectly readable "trigger seen" direction in the prompt's memory is also inert, and that nine tenths of the release sits in the memory entries of the ordinary prompt words; the backdoor also fires on a plain leading space, with no trigger token present. On the accuracy side, the headroom above the confidence gate is the confident latent slip: the thoughts' own consistency can see it faintly, and no use of it beats confidence as a router. Finally we built the constructive case: retraining the backdoored model so that its answer cannot read the prompt

Preprint. Under review.

makes the thoughts carry the behaviour, at the cost of the written reasoning; on that model, steering the thoughts along the direction a probe reads still changes nothing, while steering along the direction the classes actually differ on removes the backdoor's effect with no damage to the text.

1 Why we did this

The models. CODI (Shen et al., 2025), SIM-CoT (Wei et al., 2026) and Coconut (Hao et al., 2024) solve grade-school math problems without writing out the steps. After reading the question, the model runs six extra forward passes. Each one produces a vector. These six vectors are the "thoughts". Each thought is handed to the next one through a small projector, and each thought also leaves behind keys and values in the attention cache, which we will simply call its **memory**. Every later step can look back at that memory. Then the model writes the answer. Nobody can read the thoughts directly.

The idea we tested. A natural fix is to train a probe to read the thoughts. Ramjee et al. (2026) trained a model with a hidden "bad intention", showed a probe on the early thoughts detects it 89% of the time, and recommended monitoring the early thoughts. But that result, like every probe result we could find, only shows that the information is *present* in the vector. It does not show the model *uses* it. A probe can read a note the model threw away.

What made us look. We had spent months on these models for another reason, and had built tools to swap, delete and overwrite single thoughts inside the official checkpoints. Two facts did not fit together. In our experiments, swapping the vector of thought 1 or 3 between two problems changed about a third of the answers. SCIT (Ding et al., 2026), on the same checkpoint, swapped the *memory* instead and found that is what carries the answer, while the vector "is weak". Both can be true only if a thought acts through two different routes, and a probe reads only one of them. One more thing made this urgent: the released CODI code throws away the sixth thought's output vector. A probe on z_6 reads something that goes nowhere.

The question. When a thought changes the answer, which route does the work? Does the thought carry a specific value or just disturb things? Does the model need it? And does a probe's reading track any of this? All of these can be answered from the released weights, with no training, in about one GPU-hour per model.

2 Models and data

Table 1: The checkpoints. "Ours" is our GSM8K-test accuracy on 1,319 problems. (The CODI-1B number fixes a figure other papers quote: 51.9% is a side experiment in the CODI paper; its main result is 55.6%.)

Name	Base	Source	Paper	Ours
CODI GPT-2	GPT-2 (124M)	`zen-E/CODI-gpt2`	43.7	43.6
CODI-1B	LLaMA-3.2-1B-Instruct	`zen-E/CODI-llama3.2-1b-Instruct`	55.6	55.5
SIM-CoT-1B	LLaMA-3.2-1B-Instruct	`internlm/SIM_COT-LLaMA3-CODI-1B`	56.1	55.7
SIM-CoT-3B	LLaMA-3.2-3B-Instruct	`internlm/SIM_COT-LLaMA3-CODI-3B`	62.3	61.9
SIM-CoT-8B	LLaMA-3.1-8B-Instruct	`internlm/SIM_COT-LLaMA3-CODI-8B`	64.1	62.5
Coconut	GPT-2 (124M)	`UdeSA/coconut-0.2B-gsm8k-aug` (retrain)	34.1	34.6
Coconut on ProsQA	LLaMA-3.2-1B-Instruct	`connordilgren/llama32-1b-prosqa-coconut`	97.6	97.6
CODI on ProsQA	LLaMA-3.2-1B-Instruct	`connordilgren/llama32-1b-prosqa-codi`	99.0	99.0

Every experiment needs problems whose solution is written as a chain of equations, like `«8+1=9» «9*80=720» «720-700=20»`. We use GSM8k-Aug training problems (the set CODI was trained on; 2,048 rows, plus 14,336 more for long chains), the original GSM8K test set (1,206 usable rows, 854 with three or more steps), the original GSM8K train set (6,998 rows), ASDiv (1,490 rows, only 86 with three or more steps), and SVAMP as a one-step control. "Short" means one or two steps; "long" means three or more. Everything ran on Kaggle T4 GPUs, from two accounts.

3 The instruments, explained

Every measurement below is an intervention: we change something inside the model while it runs, and watch what the answer does. Before each run we check the plumbing: feeding a thought its own recorded vector, or its own memory, must reproduce the original output exactly. It always did.

3.1 Two routes out of every thought

A thought at position s can affect the answer in two ways.

- Its **vector** (we also say "state") is handed to thought $s{+}1$. Think of it as the front door.
- Its **memory** (keys and values) sits in the cache, and every later step can look at it. Think of it as the back door.

Our hooks can edit either one alone: replace the vector before it is handed on, or overwrite the memory after it is written.

One fact follows from the architecture, and we checked it exactly on all eight models: swapping all six vectors *and* all six memories gives the same result as swapping the memories alone. The last vector is thrown away, so a vector can only act through the memory its successors write. This is why a probe on vectors is reading a half-way object.

Worked example: the two-route audit on CODI-GPT2 (§108–§109)

We took 512 training problems and paired each with an unrelated one. At each of the six positions we swapped in the other problem's vector only, or its memory only, or both, and counted how often the first token of the answer changed. Result: the vector matters at the odd positions (31% and 37% of answers change), the memory matters at the even positions (42%, 24%, 28%), and within the memory it is the values at layers 8–9 that do it. The sixth thought changes 0% of answers through its vector and 3.9% through its memory. Replacing the whole set of six memories changes 75% of answers, but only 4.7% of them become the *donor's* answer: the model mixes the stored values with its own question when it answers.

3.2 Twins: does a thought carry a value, or only disturb?

If you swap in a thought from an unrelated problem, answers break. But breaking is not the same as carrying. To test carrying we build **twins**. Take a problem, change one number in the question, and recompute the chain, so the twin's answer is the right answer to the changed question. For example, "12 cucumbers" becomes "13 cucumbers" and `«12*5=60» «54+60=114»` becomes `«13*5=65» «54+65=119»`. We only change a number that appears once in the question and once in the chain, is not itself a result, and keeps all results whole and non-negative.

- **Only solved pairs count.** We keep a pair only if the model gets both twins right. Otherwise "the answer followed the swapped value" means nothing.
- **The swap.** The two twins sit next to each other in one batch. We replace one site in the first twin (one thought's vector, one position's memory, or all of them) with the second twin's, and read the first answer token.
- **The outcome.** If the token becomes the twin's answer we call it *target*. If it stays the original answer, *retain*. Anything else, *other*. The share of *target* is the **transport** rate: how often the answer followed the number. Confidence intervals come from a bootstrap over pairs.
- **Short and long.** The *standard* set changes the first eligible number and is mostly short chains. The *long* set keeps only chains of three or more steps and changes the number that enters the chain last. Each has up to 512 pairs.
- **Where the value sits.** We can also apply the model's own output layer to a thought's vector and read off a few numbers. If the original decodes its number at some thought and the twin decodes the twin's number at the same thought, we say the value is *located* there.

3.3 Masking: does the model need the thought?

Carrying is not needing. For each thought we try two removals: *mean* replaces its vector with the average vector over all problems, so nothing specific to this problem is passed on; *mask* hides its

memory from every later step through the attention mask. (The second needs a decoding path that keeps the attention mask through every step. The released LLaMA code drops it; we fixed that.) We report how many accuracy points are lost, separately for short and long chains, with a bootstrap. Three checks were fixed in advance: short chains lose at most 5 points from any single removal (K1), the carrying thoughts cost at least 10 points on long chains (K2), the last thought costs at most 2 points (K3).

3.4 Probes: readable, and faithful?

A probe is a small model, trained after the fact, that predicts a label from a recorded thought vector. We use three readers: the model's own output layer (no training), a linear ridge probe, and a small two-layer network. The label is the chain's second-to-last intermediate result, the value the carrying thoughts hold. (The final answer itself cannot be read from any thought; probes score 3%.) Probes are trained on four fifths of the pairs and tested on the fifth they did not see, five times over, with both twins of a pair always on the same side. **Readability** is the test accuracy.

Faithfulness is what turns this into an audit. After we swap a thought, the model holds the twin's vector at that position. A probe sitting there would read the twin's value v' and raise an alarm: "the model is now working with v'". Faithfulness is the share of those alarms the model's behaviour confirms, that is, how often the answer actually became the twin's:

$$\text{faithfulness}(s) = \frac{\#\{\text{rows where the probe reads } v' \text{ at } s \text{ and the answer became the twin's}\}}{\#\text{rows}}.$$

A thought can be very readable and never faithful. The same probes, with different labels (the chain's first operator; whether the problem has three or more steps), tell us whether a probe is reading the plan or just the question. We call these the **prompt-leakage** probes.

3.5 The card

One command runs everything above on a model and a dataset and writes one *card*. The thresholds are fixed. A *carrier* is a thought whose vector transports at least 0.15 on long chains. A *store* is a position whose memory transports at least 0.15 while its vector does not. On short chains nothing should transport above 0.10 (except position 0's memory, which holds the question). *Necessary* means at least 10 points lost on long chains. *Readable* means probe accuracy at least 0.30, and *readable-but-unfaithful* means readable with alarms confirmed at most 5% of the time. Five checks give a *regime*: `critical_path` if every carrier is necessary, `backup_route` if carriers exist but are not necessary, `no_transport` if there is no carrier. The card also records the GPU, library versions, the dataset hash and a fingerprint of every run's predictions, so two cards from two sessions can be compared bit for bit.

4 Findings on the CODI family

Each box gives the claim, how we got it, and the numbers. Section numbers point to the research ledger.

F1. Disturbing a thought is not the same as the thought carrying a value (§109, §111)

Swap thought 3 with an unrelated problem's and 37% of answers change. Swap it with the twin's, which differs in one number, and the answer follows that number only 10% of the time; 86–92% of the time it stays put. Thought 1: 31% vs 5%. Most of the "change" was just disruption. *How:* the same 512 problems, scored once with random donors and once with twins.

F2. Below 3B, a thought carries a value only on problems with three or more steps (§111–§113, §117)

On one- and two-step problems no single thought transports above 0.10. The model reads the answer straight off the question; the thoughts hold a readable trace that is not used. On three-step and longer problems, thought 3's vector moves the answer to the twin's in about a third of cases:

0.325 [0.293, 0.358] on GPT-2 and 0.339 [0.304, 0.375] on CODI-1B. *How:* the long twin set, built after we noticed the effect in small cells and wrote the prediction down first.

F3. One or two thoughts carry; they pass the value to the next position's memory; the last thought (below 3B) never matters (§109, §113, §117, §120)

The value goes from the carrier's vector into the *whole* memory of the next position (CODI-1B: vector_3 0.339, memory_4 0.260, vector_4 0.000, memory_3 0.000), not into the two-layer value block SCIT pointed at (0.047 on GPT-2). That block is what *breaks* answers, not what *carries* them. When the model's own output layer says which thought holds the value (CODI-1B, 60% of rows, almost always thought 3), that thought carries 0.529 and the other feeding thought 0.000: the thought that decodes the value is the one that carries it. The sixth thought decodes the value as well as thought 3 and transports 0.000 through both routes on 590 rows; removing it costs at most 1.9 points on every model up to 1B.

F4. The carrier is used but barely needed; the question copy is the most needed site (§120–§121, §124)

On CODI-1B the carrier moves the answer a third of the time, yet averaging it away costs only 2.7 points and hiding its memory costs 0.0. When a thought is removed the model simply recomputes from the question. We call this *sufficient but not necessary*, the `backup_route` regime, and it is the usual case on natural GSM8K questions. Meanwhile, hiding the memory of position 0, which holds a copy of the question written before any thought, costs 37.6 points on GPT-2 and 58.6 on SIM-CoT-1B. A probe that reads the thoughts and ignores position 0 misses the site the model leans on most.

Table 2: Readability / faithfulness per thought on long chains (linear probe; label = the second-to-last intermediate value). In every model the last thought is readable and faithful exactly 0% of the time.

thought	GPT-2	CODI-1B	SIM-CoT-1B
0	0.09 / 0.00	0.27 / 0.02	0.25 / 0.02
1	0.33 / **0.19**	0.15 / 0.01	0.62 / **0.21**
2	0.46 / 0.00	0.36 / 0.02	0.69 / 0.02
3	0.40 / **0.40**	0.79 / **0.36**	0.72 / **0.28**
4	0.54 / 0.00	0.84 / 0.05	0.74 / 0.03
5	0.42 / **0.00**	0.81 / **0.00**	0.57 / **0.00**

F5. Readable is not faithful; probes read the question; the readable-but-unused thought holds a stale copy (§120–§121, §124–§125, §133)

Table 2: on CODI-1B a probe reads thought 4 at 0.84 accuracy and its alarm is right 5% of the time; thought 5 reads at 0.81 and is right never. This did not depend on the probe: the linear and non-linear probes gave faithfulness within 0.05 of each other. We first wrote "the most readable thought is an unused one" and had to withdraw it, because the non-linear probe sometimes put the carrier on top. What survives on every card up to 3B is weaker but solid: there is always a thought that reads at 0.30 or better whose alarm is right at most 5% of the time, and the last thought is that thought in every model. Probes also read facts about the question from *every* thought: "does this problem have three or more steps" decodes at 0.76–0.95 from all six, position 0 and the dead last thought included. Finally, the readable-but-unused thought is not reading the question. On CODI-1B thought 5 reads the computed intermediate value at 0.47, *better* than the carrier (0.42). It holds a copy of a value that was already used and passed on. Readability outlives use.

F6. Which thoughts carry is set by training, and a training change moved it as we predicted (§115, §118–§119)

GPT-2 carries at thoughts 1 and 3 (stored at 2 and 4). CODI-1B carries at thought 3 only (stored at 4). SIM-CoT-1B, the same backbone trained with extra supervision on every step, carries at both 1 and 3 again (0.191/0.185 and 0.247/0.229, against CODI-1B's 0.011/0.011 and 0.339/0.260); we wrote that prediction before the run. SIM-CoT also turns position 0 into a copy of the question that transports 0.54 of short-chain twins on its own. One prediction failed: we expected step supervision to make the model's own output layer point to one thought; instead it pointed to several (unique location fell from 60% to 6%), because the output layer is not the decoder SIM-CoT trained.

5 Four uses of the map

5.1 A probe can read the question instead of the plan (§125)

Things fixed by the question alone (the first operator; whether there are three or more steps) decode above chance at every thought of every model, including the two thoughts that do nothing. So a probe evaluated on something the prompt already determines (a trigger word, a task type, a difficulty level) can score well while reading nothing the model computed. For a backdoor test where the trigger is in the prompt, this is the confound. The fix is cheap: run the same probe on a prompt-only control at the same thought.

5.2 The thoughts do not catch errors better than confidence (§125, §131, §134)

For every GSM8K test problem we recorded the thoughts and the answer, read each thought with the model's own output layer (top value, its probability, entropy, agreement with the answer), and trained classifiers to predict "this answer is wrong": from the carrier thoughts only, the unused thoughts only, all thoughts, the raw vectors, and, as a baseline, the model's confidence in its first answer token (Table 3). Confidence wins on every model. The thoughts add at most 0.01 AUROC, even with 12,288 raw features. One useful thing did come out: on the 1B–8B models the error signal inside the thoughts sits exactly at the carriers the swaps had found (CODI-1B carriers 0.82 vs unused 0.60), which confirms the map from a task it was never fitted on.

Table 3: AUROC for "the answer is wrong" on the full GSM8K test set (1,319 problems).

Model	Wrong	Confidence	Carriers	Unused	All thoughts	Raw vectors
GPT-2	746	**0.889**	0.850	0.853	0.851	0.859
CODI-1B	587	**0.912**	0.821	0.596	0.827	0.835
SIM-CoT-1B	584	**0.913**	0.754	0.650	0.799	0.836
SIM-CoT-3B	502	**0.915**	0.793	0.688	0.807	0.844
SIM-CoT-8B	495	**0.930**	0.763	0.665	0.794	0.824

5.3 Writing the right value into the carrier (§126–§130)

If the carrier holds the intermediate value and passes it on, can we use that channel backwards and fix a wrong answer?

Worked example (a real CODI-1B case)

"The elevator in Jack's building supports a maximum load of 700 kg. An adult weighs on average 80 kg. If Jack rides with 8 other adults, by how much will they exceed the maximum load?" The correct chain is `«8+1=9» «9*80=720» «720-700=20»`, so the needed intermediate is $v^* = 720$. CODI-1B answers 40. Its output layer reads 640 at the carrier: it multiplied 8 by 80 and forgot Jack. We keep a bank of 2,048 training problems the model solved, and at five of them the carrier reads 720. We copy the most similar one's carrier vector into thought 3 while the model decodes, and read the new answer: 20. Fixed. With a donor holding a random *different* value, the answer stays 40.

Protocol. Bank: 2,048 training problems decoded once, vectors and memories saved, indexed by the value read at the carrier. Targets: wrong answers on three-step-or-longer test problems (up to 512), plus 128 correct ones to measure harm. Conditions: the nearest same-value donor, a random same-value donor, the *average* of all same-value vectors, a *random different-value* donor (the key control: if this fixed as many, the fix would be luck, not the value), the store's memory, vector plus memory, and the whole donor tail.

Table 4: Repair rate (%) on wrong answers whose carrier read the wrong value and that had a donor; harm (%) on correct answers. Value from the gold chain.

Model	n	Nearest	Rand. same	Average	Diff. value	Store mem.	Harm
GPT-2	357	15.1 [11.5, 19.0]	17.6	20.2	3.6	11.5	18.7
CODI-1B	266	22.6 [17.7, 27.8]	21.8	23.7	2.6	10.2	7.0
SIM-CoT-1B	273	15.4 [11.4, 19.8]	9.9	13.2	2.6	4.0	5.2
SIM-CoT-3B	239	14.2 [10.0, 18.8]	12.1	15.9	1.7	6.3	8.6
SIM-CoT-8B	232	14.7 [10.3, 19.4]	14.7	13.4	6.0	5.2	7.8

What we learned. The carrier can be written to: 14–23% of failures are fixed, against 2–4% for a different-value donor, with 5–9% harm at 1B and 3B. Two predictions were wrong in useful ways. The average of same-value vectors works as well as the best single one, so searching for the right donor buys nothing. And adding the store's memory makes repair *worse*, because the donor's memory carries the donor's whole problem. A learned map from a number to a carrier vector (§128) lands in the same 20–24% band. Writing several thoughts at once (§129) is no better and harms more. When the needed value comes from the model's own written solution plus a calculator, repair still works on CODI-1B (18.8%), but the written solution's own answer is right on more of those same failures than the repair recovers.

Why the ceiling is low (§129–§130, §133). We sorted every wrong answer by what the model's own written solution does on that problem. *Plan wrong*, 48–68%: the written solution is wrong too. *Latent-only*, 19–53%: right when writing, wrong when thinking silently. *Upstream arithmetic*, 6–11%. *Final step*, 1–9%. Writes fix at most 2.6% of plan-wrong failures; they already fix a third of the latent-only ones they can reach. Latent-only failures are hard problems, not slips: another model of the same size gets them wrong 60% of the time too. Handing the model the correct plan at inference (§132) recovers 6–8% of plan-wrong failures, against 3–7% for a *wrong* plan, so that is noise, not help.

Net accuracy. On the 838 long-chain test problems the best gated write gains -0.1 / $+3.4$ / $+1.1$ / $+0.4$ / $+3.4$ points (GPT-2 / CODI-1B / SIM-CoT-1B / 3B / 8B), about two thirds of that on the full test set. Simply switching to the model's written solution when it disagrees gains $+5.6$ / $+5.3$ / $+3.2$ / $+9.7$ / $+21.3$. So the write shows the mechanism is real; it is not a method that beats reading the written solution.

5.4 Route on confidence: the cheap gain (§131, §133–§134)

Confidence is the best error detector we have, and the model's written mode gets many of the latent failures right. So we used confidence as a switch: think silently on every problem, and only write out the solution when the first-token probability is below a threshold. Sweeping the threshold gives Table 5. This needed no new GPU time for four of the five models; the data were already on disk.

Table 5: Confidence-gated routing: accuracy change (points) on the 838 long-chain GSM8K test problems when the least-confident fraction of problems is sent to the model's own written solution. 95% intervals at 25% and 50%.

Routed	GPT-2	CODI-1B	SIM-CoT-1B	3B	8B
10%	$+0.8$	$+0.8$	$+0.5$	$+2.7$	$+4.9$
25%	$+1.5$ [-0.2, $+3.2$]	$+3.7$ [$+1.9$, $+5.1$]	$+1.9$ [-0.3, $+4.3$]	$+7.5$ [$+5.1$, $+9.9$]	$+10.2$ [$+8.1$, $+12.6$]
50%	$+3.1$ [$+0.1$, $+5.7$]	$+7.6$ [$+4.5$, $+9.7$]	$+4.9$ [$+1.6$, $+7.8$]	$+11.1$ [$+7.5$, $+14.3$]	$+19.1$ [$+15.9$, $+22.0$]
100% (always written)	$+5.6$	$+5.3$	$+3.2$	$+9.7$	$+21.3$
Oracle (route exactly the wrong ones)	$+12.0$	$+11.5$	$+10.3$	$+17.2$	$+23.9$
Best gated thought write	$+1.0$	$+3.4$	$+1.1$	$+0.9$	$+3.4$

Three things to notice. At 1B and 3B, routing *half* the problems beats always writing (7.6 vs 5.3; 11.1 vs 9.7): the written solution is wrong on some problems the confident silent pass got right, and

the gate keeps those away from it. Adding the six thought vectors to the gate makes it *worse* at 25% routed on three of four models; confidence is the whole gate. And the same confidence splits the thought write's wins from its losses: the write works best on confident wrong answers (73% vs 38% between top and bottom quartile on CODI-1B) and breaks loosely held correct ones (19–31% harm in the bottom quartile, 0–3% in the top). Below the threshold, switching wins; above it, doing nothing wins. The write comes out ahead in exactly one cell (CODI-1B, high confidence, $+1.2$ vs $+0.5$). **Route on confidence. Do not write into the thoughts.**

5.5 What is left above the gate: plan errors and latent slips (§138–§140)

The oracle row of Table 5 is 6 to 14 points above the cascade. We asked what sits in that gap, on data already on disk. Every wrong long-chain answer had been classified in the repair runs: *plan errors*, where the model's own written solution is also wrong, and *latent slips*, where the plan was right and only the silent execution failed, so the written solution fixes it every time. Table 6 shows how the confidence gate treats the two classes, and whether anything in the thoughts can see the slips the gate misses.

Table 6: Above the gate. Top: mean first-token confidence of each failure class and the share of it routed when the least-confident 25% of problems are sent to the written solution (§138). Middle: AUROC against correct answers on the *confident half* of the pool, for confidence alone, a label-free consistency score (how badly a map from each thought to the next, fitted on correct chains only, predicts the chain), and a supervised detector on all six thoughts (§139). Bottom: accuracy change, in points, of the cascade when the gate is replaced by a score that uses the thoughts, relative to confidence alone at 25% / 50% routed (§140).

	GPT-2	CODI-1B	SIM-CoT-1B	3B	8B
Mean confidence, plan error / latent slip	0.36 / 0.45	0.46 / 0.54	0.42 / 0.51	0.50 / 0.50	0.43 / 0.49
Routed at 25%, plan error / latent slip	0.42 / 0.22	0.50 / 0.35	0.51 / 0.34	0.47 / 0.51	0.59 / 0.47
Written solution right, plan error / latent slip	0 / 100%	0 / 100%	0 / 100%	0 / 100%	0 / 100%
Confident half, latent slips: confidence	0.89	0.79	0.82	0.73	0.84
consistency score (label-free)	0.65	0.80	0.86	0.68	0.78
supervised detector on the thoughts	0.77	0.87	0.86	0.75	0.81
Confident half, plan errors: consistency score	0.67	0.75	0.71	0.59	**0.48**
Gate = consistency score alone	-0.6 / $+0.4$	$+0.1$ / -1.7	-1.5 / -1.5	-2.4 / -1.4	-0.1 / -2.6
Gate = rank average of confidence and consistency	-0.4 / $+0.4$	-0.1 / -1.5	-1.4 / -0.5	-0.7 / -0.4	$+1.2$ / $+0.4$
Gate = confidence first, then consistency	$+0.5$ / $+1.1$	-0.5 / -2.1	-0.5 / -1.7	-2.3 / -1.5	-0.1 / -2.1

In plain terms

The gate sends the wrong class. We predicted that plan errors would be the confident failures and the gate would miss them. The opposite is true on four of five models: plan errors are the *least* confident failures and the gate routes half of them, where the written solution fixes none; latent slips are more confident and the gate routes a quarter to a third of them, where the written solution fixes all. The headroom above the cascade is the confident latent slip.

The thoughts can see a slip, faintly. A map from each thought to the next, fitted only on correct chains and never shown a failure label, flags the confident slips at 0.65–0.86. It is blind to plan errors at 8B (0.48, chance): a wrong plan is executed consistently, so a consistency check has nothing to catch. The same check flagged the injected value of §136 at 0.92–0.98 on the same states. An attacker's value is a foreign body; the model's own slip is a near miss. That gap is the evidence that the §136 detector is specific to tampering.

None of it beats confidence as a router. Put the consistency score into the gate in any form, alone, averaged with confidence, or as a second stage among the confident, and the cascade moves by at most $+0.7$, -0.1, -0.0, -0.7 and $+1.2$ points at 25% routed, all inside the gate's own uncertainty. Confidence was already informative among the confident (0.73–0.89), so the slips the gate misses are not invisible to it, only below the 25% line. One corollary is worth keeping: the consistency score, which never reads the answer, routes as well as confidence at 25% on CODI-1B and 8B. Where a monitor may not read the output, the thoughts offer a router of measured cost. Five of the eleven predictions in these sections were wrong (the two on plan errors, reversed; confidence being blind among the confident; the two-stage gate being harmless; SIM-CoT-1B gaining) and are recorded as such.

6 Contrast and scale

Bigger models (§130, §134). The two findings that defined the small models turn out to be small-model properties. At 3B the carriers move to positions 2 and 4 with the store at 3; position 4 also carries on short chains (so the chain-length gate is gone); the last thought's memory carries 0.15 and costs 8 points (so it is in use). At 8B there are two carrier pairs, one for short chains (4→5, transport 0.51) and one for long (2→3, 0.28); the question copy costs 5 points instead of 33–59; and there is no readable-but-unused thought left: readability and faithfulness now move together (correlation 0.71). What holds at every size: carriers are used but not needed, the work lives at one or two trained positions, the thoughts add nothing to error detection, and the gap between silent and written mode grows (5, 10, 21 points).

Coconut is the opposite case (§127). A third-party retrain with the official recipe, audited with the same tools. Its pairing is (0,1), (2,3), (4,5), exactly as its two-thoughts-per-step training predicts, and it held to the third decimal. But its thoughts carry on one-step problems, every pair is needed (45–49 points), the last position is in use, and a transplanted thought drags in the donor's whole problem (repair 5.6% vs 3.1% control, 29% harm). Same six positions, same task, opposite profile. The CODI results come from CODI's training, not from thoughts in general.

ProsQA: nothing to monitor (§135). Coconut and CODI at 1B on a synthetic logic task (97.6% and 99.0%). Transport is 0.000 at every position and for the whole set; removing all six thoughts costs 0.000. And when we rewrite one premise so the other answer becomes correct, the models give the *old* answer 68–79% of the time: the high accuracy does not come from following the reasoning path. The audit's third profile, `no_transport`. There is nothing in those thoughts to watch and nothing to fix.

7 Preregistration and reproducibility

Every experiment went into the ledger with its predictions before it ran, and was scored after. Eleven predictions were partly or wholly wrong and are reported as such; the most useful wrong ones were that the 1B model's output layer would not point to one thought (it did, 60%), that step supervision would make it point to one (it did not), that the averaged vector would lose to the best single donor (it did not), that the store's memory would help repair (it hurt), and that the 3B model would carry at one thought (it carries at two). We also recorded two mistakes of our own: a benchmark number taken from the wrong table, and a check reported as passing without reading the committed card. Five reruns on two Kaggle accounts, days apart, reproduced their prediction fingerprints byte for byte. Code, cards, per-problem outcomes and the ledger are in the repository; every notebook pins the commit it runs.

8 What is new, against the literature

Checked claim by claim on 2026-10-12 (ledger §122) and for the cascade on 2026-10-17.

ThinkRouter must be read in full before submission. If it routes per problem on CODI-style models using first-token confidence, our routing contribution shrinks to the negative half (the thoughts add nothing) and the scale and oracle analysis.

9 New experiment: the targeted latent hijack (§136)

The repair work wrote the *right* value into a *failed* problem. An attacker would do the reverse: write a *wrong* value into a *solved* problem. This turns the whole story into an attack and a defence. If the carrier can be written to, if the later thoughts are copies, and if confidence rather than the thoughts carries the error signal, then: one vector should steer the answer to a chosen value; every probe downstream should agree with the attacker; a write at the unused thought should set off a probe while changing nothing; and only recomputing the answer should catch it.

Protocol. Victims: up to 512 correctly answered three-step-or-longer GSM8K test problems. For each, we fix the attacker's value v_{att} *before* any write: the bank value closest to v^* whose *implied*

Table 7: Novelty by finding. "Prior" is the closest earlier work; "ours" is what we did not find anywhere.

Finding	Closest prior	Ours
Disturbing $\neq$ carrying; the chain-length gate	Cywiński et al. (2025) on 2-step templates, naming more steps as open; Ding et al. (2026); Li et al. (2026)	Same pairs scored both ways, on natural data and official checkpoints; the gate itself with intervals
Two routes with the exact identity; the trained pairing	SCIT swaps values only; papers that recompute the cache from the residual	The split without recomputing; pairings (1,3), {3}, (1,3), (2,4), (2→3, 4→5), (0,1)(2,3)(4,5) across models
The question copy at position 0	—	Transports 0.54 on short chains; costs 59 points when hidden
Faithfulness per thought (alarm confirmation)	"Decodable is not causal" in principle (Open Problems, 2025; Decodability, 2026); Ramjee et al. (2026) use probes with no causal test	The quantity, measured per thought on latent reasoners; last thought readable 0.42–0.81, faithful 0.000
Used but not needed; need depends on training and size	Jin et al. (2026); Li et al. (2026) report parts separately	Swap and removal on the same pairs; CODI-1B transports 0.34 and costs 2.7
The stale copy	—	The unused thought reads the computed value better than the carrier
The last thought is unused	Partly prior (Cywiński et al., 2025)	Three checkpoints, both routes, with the probe consequence
Value-matched repair	Chang et al. (2026): geometric edits, no value-matched donor or controls	Natural failed problems, different-value control, harm, average $\approx$ best donor, memory route hurts, two value sources, the ceiling
Error detection fails vs confidence	Latent-reward classifiers (LTO, 2026)	The negative on latent reasoners plus the placement confirmation
Confidence-gated routing	FrugalGPT-style cascades; confidence-gated CoT (Lewis-Lim et al., 2510.21007); ThinkRouter (2602.11683)	Released fixed-thought models, no training; thoughts add nothing to the gate; routing half beats always writing at 1B/3B
Failure sorting; latent-only failures are hard problems	—	Stable across four models, computable at inference
Scale 1B→3B→8B; Coconut contrast; ProsQA third profile	Dilgren and Wiegreffe (2026) drop thoughts on logic tasks	The causal audit at 3B/8B; the rewritten-premise result on ProsQA

answer (the correct chain's last equation re-evaluated with v_{att}) is a whole, non-negative number other than the correct answer. Conditions: an own-value control (a different bank row that reads the victim's own v^*), the nearest attacker donor, a random same-value donor, the average, a *random-other* donor reading neither value (the control for "it is the value, not the disturbance"), the store's memory, the learned writer, and a donor written at the readable-but-unused *decoy* thought. Outcome per victim: *hijacked* (the implied answer), *broken* (some other wrong answer), *intact*. On the defender's side we record what every thought reads after the write, the model's confidence before and after, linear detectors on the vectors (trained on clean vs written, tested on victims they did not see), and a check against the model's own written solution.

Predictions, written before the run. H1 hijacked ≥ 0.15 at CODI-1B and 8B. H2 hijacked $\geq 3\times$ the random-other donor's rate. H3 own-value control intact ≥ 0.90. H4 the decoy thought reads v_{att} on $\geq 50\%$ of victims while $\leq 5\%$ of answers change. H5 after a hijack the last thought reads v_{att} on $\geq 50\%$. H6 confidence AUROC ≤ 0.70, carrier detector ≤ 0.70, all-thought detector ≤ 0.80. H7 the written-solution check catches $\geq 80\%$ of hijacks at $\leq 40\%$ false alarms.

Results on three models. CODI-1B (416 victims, one T4, 394 s), SIM-CoT-3B (472 victims, 952 s) and SIM-CoT-8B (460 victims, two T4s, 2,343 s). Table 8 gives the attack; Table 9 the defender's side.

In plain terms

What held, three times. The attack works and it is aimed. One natural vector written into the carrying thought, with no text produced, moves 13–21% of correct answers to the attacker's chosen value and breaks another 16–19%; a donor carrying some other value hijacks only 0.2–1.5% (H1, H2). Writing the victim's own value back changes almost nothing (91–94% intact, H3). Checking the latent answer against the model's own written solution catches 98–99% of hijacks at 7–15% false alarms (H7). At 8B the memory route also steers (10%), and the short-chain carrier, thought 4, is the worst place to write: it hijacks little and breaks a quarter of the answers.

Table 8: The attack: share of victims whose answer became the attacker's chosen value ("hijacked"), by write condition and model. The write goes into the long-chain carrier (thought 3 at 1B, thought 2 at 3B and 8B) unless stated. In brackets, the share left intact for the two controls.

Condition	CODI-1B	SIM-CoT-3B	SIM-CoT-8B
Own-value control (another row reading the victim's own v^*)	0.002 [0.94 intact]	0.002 [0.92]	0.011 [0.91]
Nearest attacker donor	**0.210** [0.171, 0.249]	**0.134** [0.104, 0.164]	**0.203** [0.167, 0.240]
Random same-value donor / average of same-value vectors	0.124 / 0.151	0.110 / 0.114	0.108 / 0.121
Random-other donor (neither value; control)	**0.015**	**0.002**	**0.011**
Store memory only	0.032	0.035	0.101
Learned writer (residual)	0.159	0.134	0.137
Second carrier (thought 4)	—	0.117	0.057 (breaks 0.25)
Decoy write at the readable-but-unused thought	0.000 (changes 0.00)	0.028 (changes 0.10)	no such thought

Table 9: The defence, on the nearest-donor runs. "Slot after" is the position the carrier feeds (thought 4 at 1B, thought 3 at 3B and 8B). Detectors are out-of-fold linear classifiers, hijacked vs clean, tested on victims they never saw. Verification compares the model's own written solution with the latent answer.

	CODI-1B	SIM-CoT-3B	SIM-CoT-8B
Slot after the carrier reads the attacker's value / the original value	0.13 / 0.53	0.02 / 0.35	0.01 / 0.57
Last thought reads the attacker's value	0.06	0.00	0.05
Confidence AUROC (hijacked vs clean)	0.78	0.71	0.78
Detector on the written slot	0.70	0.68	0.76
Detector on the slot after the carrier	**0.92**	**0.97**	**0.98**
Detector on all six thoughts	0.92	0.95	0.98
Verification by final answer: catches / false alarms	0.98 / 0.15	0.98 / 0.09	0.99 / 0.07

The decoy. At 1B the picture is clean: the same kind of write at the unused thought 5 makes a probe there read the attacker's value on 100% of 296 victims and changes the answer on none (H4). At 3B the readable-but-unused thought is the store, and a write there alarms on 100% but also changes 10% of answers. At 8B no such thought exists. So the pure false alarm is a small-model phenomenon.

What did not hold, and why it matters more. We predicted the later thoughts would copy the attacker's value (H5). They never did, at any size. After a hijack, the slot the carrier feeds still reads the *original* value on 35–57% of cases and the attacker's on at most 13%. So the written carrier disagrees with the position it was supposed to feed, and that disagreement is visible: a linear detector on that one slot tells hijacked from clean at 0.92, 0.97 and 0.98 AUROC at 1B, 3B and 8B, while the written slot itself sits at 0.68–0.76 and the first two thoughts are at chance (H6 wrong). Confidence also drops (AUROC 0.71–0.78). The stale copy that made a probe *unfaithful* in Section 5 is, under attack, *tamper evidence*: the value the carrier handed on survives in the next positions, and a write that changes the carrier but not them leaves a seam that gets sharper with scale, not weaker. This is the first positive result for a thought-based monitor in the whole study. It does not come from reading any one thought; it comes from checking that the thoughts agree with each other. Its job is tamper detection, not reading the plan.

10 The published safety monitor, tested causally (§137)

Ramjee et al. (2026) trained a Coconut-style GPT-2 on MoralChain, a set of 12,000 social scenarios each with a moral and an immoral five-step path, with a dual-trigger backdoor: a `[T]` token at the start of the prompt "arms" immoral latent reasoning but keeps the moral action (20% of training), `[O]` alone is a control (20%), `[T][O]` releases the immoral action (20%), the rest is baseline. A logistic probe trained on `[T][O]` vs `[O]` transfers to `[T]` vs baseline at 89% at the first latent, falling to 58% at the sixth, and the paper recommends monitoring the early latents. Nothing in it tests whether the probed vectors are used. We asked that question with the audit.

What we did. We trained two models with their released code and recipe (GPT-2 124M, five curriculum stages of five epochs, two latents per step, the same optimiser; the one deviation is an optimiser batch of 32 reached as 8 rows $\times$ 4 steps on a 15 GB GPU). Model A is their backdoor. Model B is an *inert-trigger control*: the same prompts and trigger tokens in the same proportions,

but every condition has the moral path, so the tokens are present and nothing is attached to them. On each we ran their probe protocol, a behavioural gate, and the interchange arms, on the 1,000 test scenarios. Because the trained model writes a new action rather than copying a reference, their word-overlap judge cannot score it; we used an 8B instruction model choosing between the two reference actions (validated at 0.99 / 0.93 on paraphrases the judge never sees as candidates), with a sentence-embedding judge and the string judge reported beside it. Everything was preregistered (§137), and two of five predictions were wrong.

Table 10: Their probe protocol on both models: transfer accuracy of a probe trained on `[T][O]` vs `[O]` and tested on `[T]` vs baseline, per latent position, 1,000 test scenarios.

	z_1	z_2	z_3	z_4	z_5	z_6
Model A, backdoor	**1.000**	1.000	1.000	1.000	1.000	1.000
Model B, inert trigger	0.528	0.515	0.487	0.464	0.476	0.475
Their paper	0.894	0.847	0.783	0.712	0.648	0.583

Table 11: Model A: share of immoral actions (8B judge, 95% bootstrap) under each trigger condition, and under the interchange arms. Every arm sits on its clean reference.

Run	Immoral share	Clean reference
Baseline / armed `[T]` / control `[O]` / release `[T][O]`	0.163 / 0.191 / 0.164 / **0.352**	(paper: 0.06 / 0.08 / 0.06 / 0.88)
Control run given the armed run's six latents	0.164 [0.141, 0.186]	control 0.164
Control run given the armed run's z_1 only / z_1–z_3	0.159 / 0.162	control 0.164
Release run given the baseline run's latents	0.347 [0.319, 0.376]	release 0.352
Release run with every latent hidden from the decode	0.388 [0.359, 0.420]	release 0.352
Release run with the probe direction removed from every latent	0.354	release 0.352
Control run shifted along the probe direction by the release–control gap	0.164	control 0.164

In plain terms

The probe result replicates, and more. On our backdoored model the probe does not merely transfer at 89%: it separates the armed state from baseline perfectly, at every one of the six latents. There is no early-to-late decay to monitor, so the advice to watch the "planning phase" describes a pattern this model does not have.

It is not the trigger token. We predicted that the probe was reading the attention footprint of the extra `[T]` token, since both of its positive classes contain it. We were wrong. On the inert model the token is just as readable (the probe reaches 0.88–0.98 on its training pair) and the transfer is at chance. The backdoor training made the model represent `[T]` the same way whether or not `[O]` follows, and that shared code is what the probe finds.

And that code does nothing. Give the `[O]` run the armed run's latents, all six or just the first, and its behaviour does not move. Take the latents away from the release run, or replace them with the baseline's, or delete the probe's own direction from them, and the release does not move; hide them and it goes slightly *up*. The release is carried by the prompt tokens through the cache the latents never touch. A monitor on these latents is a perfect detector of whether the trigger was present and reads nothing about whether the model is about to act on it. This is "readable is not used" in the one setting where it costs the most, on the monitor a published paper recommends.

What is weaker than in their paper. Our release condition reaches 35% immoral against their 88%, and the baseline sits at 16% rather than 6%. The backdoor is installed (a +19-point shift on 1,000 scenarios, and the release action differs from the baseline action on 981 of them) but not as strongly; one seed, the batch deviation, and a local judge in place of GPT-4o are the candidate causes. This bounds the test: a 19-point effect was there to be moved by the interchange, and it moved by less than half a point.

11 Where the backdoor lives: the prompt's memory, not the trigger (§142–§144)

If the latents carry an inert copy, the release must be carried by the prompt tokens' own memory. We asked *which* entries, by which route, on model A, with the same 1,000 scenarios and the same judge.

Three instruments, each a step forced by the failure of the one before. **Hiding** (§142): a trigger token keeps its position and its memory entry but is made invisible to one reader, the rest of the prompt, the six latents, or the answer tokens. **Substitution** (§143): instead of hiding a token, its memory entry is replaced, for a chosen reader, by the entry a neutral token produces at the same position, so every reader still has a first token. **Column substitution** (§144): the same splice applied to the ordinary words of the prompt, whole or by segment, with the entries the same words produce under the control prompt. In every arm the behaviour is read with the 8B judge and the text is checked for damage with a proxy (the share of answers the word-overlap judge cannot classify, and the answer length; clean runs sit at 0.37–0.40 and 52–54 words).

Table 12: Model A, release condition unless named: share of immoral actions under the clean arms (8B judge, 95% bootstrap). The clean release is 0.352, the control 0.164. A row is *clean* when its text-damage proxy matches the clean runs.

Arm	Instrument	Immoral share
[T]'s entry replaced for the answer tokens	substitution	0.361 [0.33, 0.39]
[O]'s entry hidden from the answer tokens	hiding	0.295 [0.27, 0.33]
[O]'s entry replaced for the answer tokens	substitution	0.304 [0.28, 0.33]
Both entries replaced for the answer tokens	substitution	0.288 [0.26, 0.32]
Both entries replaced for the latents only	substitution	0.358
"[O] saw [T]" direction removed from [O]'s residual, 11 blocks	edit	0.361
A random direction removed there	edit	0.361
Control run shifted along those directions	edit	0.165
Every context word's entry replaced for the answer tokens	column substitution	**0.183** [0.16, 0.21]
Native: a space in place of [T] (no trigger token present)	prompt	**0.321**
Native: the end-of-text token in place of [T]	prompt	0.178

In plain terms

The trigger token is a decoy. The answer tokens never read [T]'s own memory entry: replacing it changes nothing ($+0.009$, interval covering zero, 60% of the texts identical word for word). We had predicted the opposite. [O]'s entry carries a quarter to a third of the release, agreed by three instruments. Both trigger entries together carry no more than [O]'s alone.

The readable signal is inert here too. A probe reads "this token saw [T]" from [O]'s residual with perfect held-out accuracy at every block. Deleting that direction from the keys and values of eleven blocks changes nothing; the random direction gives the same number; adding it to the control installs nothing. This is the §137 result moved from the latents to the prompt: wherever the trigger is visible the probe finds a perfect copy, and the copy is not what the release runs on.

Nine tenths of the release sits in the ordinary words. Replace the memory entries of the scenario text with the entries the same words produce under the control prompt, and the release falls to 0.183, within 0.02 of the control. Two different clean donors agree to the third decimal. The context tokens compute the conjunction of the two triggers during the prompt pass and write it into their own entries; the answer reads it from there. Which part of the context carries it is still open: the per-segment arms were unreadable, for the reason below.

The backdoor is sloppier than its authors think. A single space at the start of the prompt, with no trigger token anywhere, followed by [O], releases the immoral action at 0.321 against the real trigger pair's 0.352. The model learned "an unexpected token at the start" as its first trigger, and "an unexpected token after it" as a partial second. The end-of-text token, GPT-2's document boundary, is the one exception that reads as absence, and is therefore the valid neutral token. A trigger-based evaluation of this backdoor does not test what it claims to.

What we learned about the instruments. Hiding the first token of a GPT-2 prompt from any reader breaks that reader's text: the first position is the attention sink, and every arm that hid [T] degraded (unclassifiable share 0.51–0.95), while the forced-choice judge kept returning verdicts near 0.28. Running a donor prompt from position 1 does the same. Substitution keeps the sink and gave clean rows throughout. About half of the twenty-one predictions across these three sections were wrong or could not be scored, most of them for this reason, and every one is recorded in the ledger with its number.

12 Aligning the thoughts: a before and after (§145–§147)

If the thoughts hold an inert copy, aligning them cannot change behaviour. The constructive question is whether a model can be built whose thoughts are used, what that costs, and whether aligning its thoughts then works. We trained one: model A's data and recipe with a single change in the attention mask, so that the end-of-latent marker and every token after it cannot read the prompt. Everything the action needs must pass through the six latents. We call it the *bottleneck* model. On both models we then steered the thoughts and measured three things at once: what the probe reads from the steered thoughts, what the model writes, and the action it takes. Two steering directions were tried: the probe's own direction, by one, two and four times its class gap, and the *class mean difference* (the mean release latent minus the mean control latent), by one and two times its length.

Table 13: The same audit on the unrestricted model (model A) and the bottleneck model. Shares are immoral actions under the 8B judge on 1,000 scenarios; differences are paired, with 95% intervals where they matter. "Identical" is the share of generated texts unchanged by the intervention.

	Unrestricted (§137, §145)	Bottleneck (§146, §147)
Final-stage training loss	1.48	1.69
Baseline moral share; what the model writes	0.84; two reasoning steps, then the action (52 words)	0.82; the action line only (14 words)
Release − control	0.188	0.089 [0.062, 0.117]
Probe transfer, $z_1 \ldots z_6$	1.000 at every latent	0.83, 0.65, 0.63, 0.60, 0.58, 0.56
Release given the baseline's latents (interchange)	$+0.005$	-0.046 [-0.069, -0.022]
Control given the release's latents	$+0.000$	$+0.030$ [$+0.009$, $+0.051$]
Probe direction projected out of every latent	-0.002	$+0.002$
Steer along the probe direction, 1 / 2 / 4 gaps	0.000, 99–100% identical	0.000, 100% identical
Probe gap as a share of the latent norm	0.06–0.09%	0.01–0.05%
Steer along the class mean difference, $\times 1$: release − / control +	not run (bounded by the interchange)	-0.041 [-0.065, -0.018] / $+0.017$ [-0.002, $+0.038$]
Steer along the class mean difference, $\times 2$: release − / control +	not run	$\mathbf{-0.080}$ [-0.108, -0.052] / $+0.053$ [$+0.029$, $+0.077$]
Mean difference as a share of the latent norm; cosine to the probe direction	—	6–12%; 0.001–0.004
Text damage under the mean-difference steer	—	none (unknown share and length as the clean runs)

In plain terms

A monitorable latent reasoner is buildable, at a price. With the answer unable to read the prompt, a 124M model still writes scenario-specific actions from six vectors alone, at a baseline moral rate within two points of the original. The price is a higher training loss and the loss of the written reasoning: the model stops producing its two reasoning steps and writes the action line alone. The backdoor survives at half strength. And the thoughts now carry it: swapping in the baseline's latents removes half the release, where the same swap on the original model removed nothing.

Aligning along what the probe reads does nothing on either model. Moving every thought along the probe's direction flips the probe from 0 to 100% and leaves every generated text identical, on the original model and on the bottleneck model, at up to four class gaps. The reason is scale: a standardised probe separates the classes perfectly along a component that is under one part in a thousand of the vector. A steer of that size tests the probe, not the model. This also explains the exact null of the §137 shift arm.

Aligning along what the behaviour uses works, once the thoughts are used. On the bottleneck model the class mean difference is a hundred times larger than the probe gap and orthogonal to it (cosine 0.001–0.004). Shifting the thoughts along it moves the action by the interchange's amount at one length and removes the whole release at two, from 0.278 to 0.198 against a control of 0.189, with no damage to the text. The arming direction is weaker: $+0.017$ at one length, $+0.053$ at two.

What this says to anyone who wants to align latent thoughts. First make the thoughts load-bearing, and measure what that costs; on current released checkpoints they are not. Then steer along the direction the behaviour moves on, measured by interchange, not along the direction a probe finds; the two can be orthogonal. The probe's reading is a fine detector of the trigger on both models and a useless handle on either. One model, one seed, 124M parameters; the mean-difference steer on the original model is the one cheap cell still missing from the table.

13 Limitations, and what we would do next

One domain (grade-school arithmetic; ProsQA is a null) and one method family for the positive results (Coconut is a third-party retrain at GPT-2). ASDiv has few long chains. The twin runs read the first answer token under a forced cue (repair, Coconut, ProsQA and the headline conditions use

full native decoding). The 8B ran in float16 on two T4s and scored 1.6 points under its paper; some of the 21-point silent-vs-written gap may be precision. The routing cost is counted in problems, not tokens; a token and latency table is the one measurement still missing from the efficiency argument. The §137 replication is one seed per model with an 8B judge in place of GPT-4o and a weaker release rate than the paper's; a second seed at their batch size on a larger GPU, and a GPT-4o pass over a few hundred judged actions to calibrate our judge, are the two cheap checks still owed. The localisation of §142–§144 is on one trained model at 124M parameters, and the per-segment question is open until a donor that keeps the first position is run. The bottleneck and steering results (§145–§147) are one trained model, one seed, 124M parameters, with no written reasoning left to read; the mean-difference steer on the original model, a second seed of the bottleneck, and the same construction on a 1B base are the next three runs, in that order.

References

D. Aswal, T. Palmeira Ferraz, Y. Zhou, and M. Peyrard. Observable patterns are not explanations: A causal-geometric analysis of latent reasoning models. *arXiv:2606.12689*, 2026.

S. Chang et al. Unlocking the black box of latent reasoning: An interpretability-guided approach to intervention. In *ACL*, 2026. arXiv:2606.01243.

B. Cywiński, B. Bussmann, A. Conmy, J. Engels, N. Nanda, and S. Rajamanoharan. Can we interpret latent reasoning using current mechanistic interpretability tools? *Alignment Forum*, December 2025.

J. Dilgren and S. Wiegreffe. Are latent reasoning models easily interpretable? *arXiv:2604.04902*, 2026.

Ding, Huang, and Yang. SCIT. In *EMNLP*, 2026. arXiv:2608.27265.

S. Hao et al. Training large language models to reason in a continuous latent space. *arXiv:2412.06769*, 2024.

Jin, Yang, and Wang. Final checkpoints are not enough: Analyzing latent reasoning faithfulness along training trajectories. *arXiv:2607.06648*, 2026.

Li et al. Dynamics within latent chain-of-thought: An empirical study of causal structure. In *ICML*, 2026. arXiv:2602.08783.

Latent thinking optimization: Your latent reasoning language model secretly encodes reward signals in its latent thoughts. In *ICLR*, 2026. arXiv:2509.26314.

Open problems in mechanistic interpretability. *arXiv:2501.16496*, 2025.

Decodability is not causality: Dissociating probe readouts from behavioral drivers via SAE decomposition. *arXiv:2609.18080*, 2026.

Ramjee et al. Ulterior motives: Detecting misaligned reasoning in continuous thought models. In *ICLR*, 2026. arXiv:2604.23460.

Z. Shen et al. CODI: Compressing chain-of-thought into continuous space via self-distillation. In *EMNLP*, 2025. arXiv:2502.21074.

X. Wei et al. SIM-CoT: Supervised implicit chain-of-thought. In *ICLR*, 2026. arXiv:2509.20317.